

PHARMAGARDE CM

Documentation des workflows – Version 2.0 (méthodologie alignée POSTrack)

Projet : PharmaGarde CM

Périmètre : flux opérationnels, données, validation, gestion des incidents

Date : septembre 2026

1.2 Historique de la mise à jour

Version	Contenu	Statut
1.0	Workflows initiaux : utilisateur, pharmacie, administration ; confirmation, signalement, incident critique ; stack technique indicative FastAPI/Python.	Archivée
2.0	Formalisation des exigences fonctionnelles transverses (F-0x), matrice de responsabilités par rôle, précision des contrôles serveur à chaque étape. Chaîne technique reformulée sur Node.js .	Version courante

1.3 Préambule

Cette documentation décrit le comportement attendu de PharmaGarde CM du point de vue des utilisateurs et du métier. Elle ne constitue pas une spécification technique détaillée, mais impose les règles fonctionnelles que l'API, la base de données et l'interface doivent respecter.

Principe directeur : toute information affichée comme « garde active » doit être rattachable à un `DutySchedule` valide, et toute confirmation ou signalement doit être horodaté et traçable jusqu'à sa résolution.

1.4 Workflow global

Le système repose sur trois flux principaux : utilisateur, pharmacie et administration.

```
UTILISATEUR → ouverture → localisation/ville →  
recherche → pharmacies de garde  
           → filtre/proximité → fiche →  
appel/itinéraire → (éventuel) signalement
```

```
PHARMACIE → connexion → consultation planning →  
début de garde → confirmation  
           → publication du statut →  
réception/traitement des alertes → fin de garde
```

```
ADMINISTRATION → collecte → validation →  
publication → surveillance  
           → traitement des signalements →  
audit → correction
```

1.5 Workflow utilisateur détaillé

Étape	Action	Traitement système
1	Accès	L'utilisateur ouvre l'application.
2	Localisation	Il autorise la localisation ou choisit une ville/quartier.
3	Recherche	Le système recherche les gardes valides correspondant à la date/heure.
4	Classement	Les résultats sont triés par proximité et fiabilité.
5	Consultation	L'utilisateur ouvre la fiche d'une pharmacie.
6	Vérification	Il voit le planning, le statut de confirmation et la dernière mise à jour.
7	Action	Il appelle ou demande l'itinéraire.
8	Signalement	S'il constate une anomalie, il peut envoyer un signalement.

1.6 Workflow de confirmation de garde

La confirmation est un mécanisme de confiance, pas une preuve absolue d'ouverture. Elle doit donc toujours être accompagnée d'un horodatage et d'un statut clair.

Planning valide → notification à la pharmacie → pharmacie ouvre son espace
→ bouton « Confirmer la garde » → serveur enregistre date/heure
→ statut publié → compteur de fraîcheur de confirmation → rappel si nécessaire

Règle	Comportement attendu
Confirmation reçue	Statut « confirmée récemment », horodatage affiché.
Confirmation absente, garde théoriquement en cours	Statut dégradé (« ancienne » / « non confirmée »), jamais « fermée » automatiquement.
Confirmation hors fenêtre de garde	Rejetée par le serveur, journalisée comme anomalie.
Rappel non traité	Nouvelle notification à l'approche de l'échéance de garde.

1.7 Workflow des signalements

Signalement utilisateur → création du ticket → classification → contrôle automatique
→ notification à la pharmacie → réponse pharmacie ou vérification admin
→ décision → mise à jour des données → clôture et journalisation

Statut	Signification
Nouveau	Signalement reçu mais non traité.
En vérification	Une vérification est en cours.
Confirmé	Le problème est considéré comme réel.
Rejeté	Le signalement n'est pas suffisamment fondé.
Résolu	Les données ont été corrigées.

1.8 Workflow de gestion d'une pharmacie

Demande d'inscription → collecte des informations
 → vérification
 → validation administrative → compte
 professionnel activé
 → pharmacie gère ses informations → planning de
 garde → confirmations → surveillance

Aucun compte pharmacie n'accède à l'espace professionnel avant validation administrative explicite.

1.9 Workflow de programmation des gardes

Source du planning → import/saisie → contrôle
 des doublons et conflits

→ validation → publication → notification aux pharmacies concernées
→ disponibilité dans l'application

Un `DutySchedule` en conflit d'horaire avec un planning existant pour la même pharmacie est rejeté par le service métier avant toute publication.

1.10 Workflow de données et fiabilité

Chaque information critique doit conserver : source + date de collecte + date de dernière modification + acteur de modification + statut de vérification.

Niveau	Exemple	Affichage
Vérifiée	Donnée validée par administration/source reconnue	Fiabilité élevée
Confirmée récemment	Pharmacie a confirmé sa garde	Confirmation récente
Ancienne	Pas de mise à jour récente	Avertissement
Signalée	Anomalie rapportée	À vérifier

1.11 Workflow technique

```
Client → HTTPS → API Node.js (contrôleur)
  → authentification/autorisation (JWT + rôle)
  → service métier
  → PostgreSQL/PostGIS (via ORM)
  → cache éventuel (Redis)
  → réponse JSON → interface utilisateur
```

Pour la proximité :

```
Position GPS → requête géospatiale PostGIS →
  filtre sur pharmacies de garde
  → calcul de distance → tri → résultats
```

Le contrôleur Node.js ne doit exécuter aucune règle métier directement : il valide la requête, transmet au service correspondant, et restitue la réponse formatée — même séparation contrôleur/service que dans l'architecture POSTrack (routes FastAPI / services métier).

1.12 Workflow d'incident critique

Si plusieurs utilisateurs signalent une pharmacie fermée :

```
Accumulation des signalements → augmentation de
  la priorité → notification à la pharmacie
  → vérification → (si nécessaire) suspension
  temporaire de l'indication de disponibilité
  → information à l'utilisateur que la donnée est
  en vérification → correction après validation
```

Le seuil de bascule automatique en « à vérifier » est un paramètre de configuration côté back-office, ajustable par ville

sans modification de code.

1.13 Workflow quotidien d'exploitation

Moment	Processus
Avant la garde	Planning chargé et vérifié.
Début de garde	Notification + possibilité de confirmation.
Pendant la garde	Surveillance des confirmations et signalements.
Fin de garde	Expiration automatique du statut.
Chaque jour	Contrôle des anomalies, sauvegardes et rapports.
Chaque semaine	Analyse des statistiques et de la qualité des données.

1.14 Exigences fonctionnelles transverses

Identifiant	Exigence
F-01	Une pharmacie n'est affichée comme garde active que si elle correspond à un <code>DutySchedule</code> valide pour la date/heure courantes.

Identifiant	Exigence
F-02	Toute confirmation est horodatée côté serveur et non modifiable a posteriori.
F-03	L'absence de confirmation ne doit jamais entraîner un affichage automatique « fermée » – uniquement une dégradation du niveau de fiabilité.
F-04	Tout signalement doit passer par un contrôle de classification avant impact sur l'affichage public.
F-05	Toute donnée critique conserve sa source, sa date de dernière modification et son acteur de modification.
F-06	Le serveur doit révérifier le rôle et l'appartenance de la ressource sur chaque opération d'écriture, indépendamment des contrôles frontend.
F-07	Toute action administrative sensible (validation, suspension, modération) doit être journalisée dans AuditLog .
F-08	Un DutySchedule en conflit d'horaire avec un planning existant de la même pharmacie doit être rejeté avant publication.
F-09	Un cumul de signalements convergents sur une même garde doit déclencher automatiquement le statut « à vérifier ».
F-10	Les erreurs de validation (import, formulaire) doivent être restituées de manière exploitable par l'utilisateur ou l'administrateur.

1.15 Responsabilités par rôle

Rôle	Consultation	Création / mise à jour	Validation / administration
Visiteur / Utilisateur	Pharmacies de garde publiques de sa zone de recherche.	Peut créer un signalement.	—
Pharmacie	Son propre profil, planning et historique de confirmations.	Peut mettre à jour son profil, son planning et confirmer une garde.	Peut répondre à un signalement la concernant.
Administrateur	Pharmacies, gardes et signalements de sa/ses ville(s).	Peut programmer une garde et corriger des données de sa zone.	Peut valider un compte pharmacie et modérer un signalement de sa zone.
Super administrateur	Plateforme entière, toutes villes.	Peut agir sur toute ressource.	Gère les rôles, les paramètres globaux, l'audit et l'arbitrage inter-villes.

Le contrôle de rôle est complété par le contrôle de périmètre géographique : un administrateur de ville ne peut pas valider ou modérer une ressource rattachée à une autre ville tant que le super administrateur ne lui en a pas étendu l'accès.

*Fin de la documentation des workflows – PharmaGarde CM
v2.0*